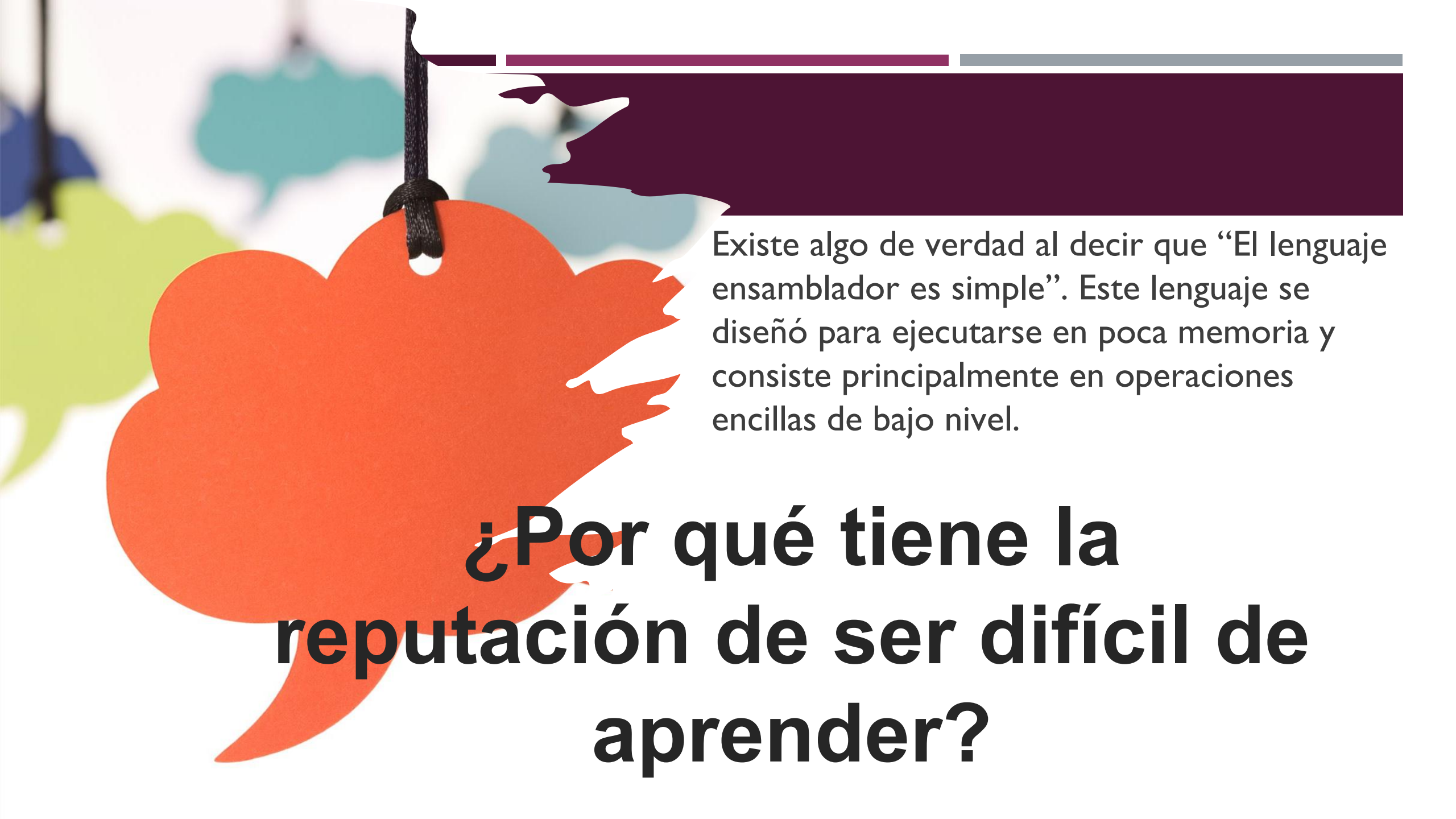




FUNDAMENTOS DEL LENGUAJE ENSAMBLADOR



Existe algo de verdad al decir que “El lenguaje ensamblador es simple”. Este lenguaje se diseñó para ejecutarse en poca memoria y consiste principalmente en operaciones sencillas de bajo nivel.

**¿Por qué tiene la
reputación de ser difícil de
aprender?**

```
main PROC
    mov     eax,5           ; mueve 5 al registro EAX
    add     eax,6           ; suma 6 al registro EAX
    call    EscribeInt      ; muestra el valor en EAX
    exit                    ; termina
main ENDP
```

¿QUÉ TAN DIFÍCIL PUEDE SER MOVER DATOS DE UN REGISTRO A OTRO Y REALIZAR UN CÁLCULO?

CONSTANTES ENTERAS

- Una *constante entera* (o literal entera) está compuesta de un signo opcional a la izquierda, uno o más dígitos y un carácter opcional de sufijo (llamado *raíz*) que indica la base numérica:

`[{+|-}] dígitos [raíz]`

- La *raíz* puede ser una de las siguientes (en mayúsculas o minúsculas):

h	Hexadecimal	r	Real codificado
q/o	Octal	t	Decimal (<i>alternativo</i>)
d	Decimal	y	Binario (<i>alternativo</i>)
b	Binario		

Si no se da una raíz, se asume que la constante entera es decimal. He aquí algunos ejemplos que utilizan distintas raíces:

26	Decimal	42o	Octal
26d	Decimal	1Ah	Hexadecimal
11010011b	Binario	0A3h	Hexadecimal
42q	Octal		

- Una constante hexadecimal que empieza con una letra debe tener un cero a la izquierda, para evitar que el ensamblador la interprete como un identificador.

EXPRESIONES ENTERAS

- Una *expresión entera* es una expresión matemática que involucra valores enteros y operadores aritméticos. La expresión se debe evaluar como un entero, el cual puede almacenarse en 32 bits (del 0 al FFFFFFFFh).
- En la siguiente table se presentan los operadores aritméticos de acuerdo con su orden de precedencia, de mayor (1) a menor (4).

Operador	Nombre	Nivel de precedencia
()	Paréntesis	1
+, -	Unario positivo, unario negativo	2
*, /	Multiplicación, división	3
MOD	Módulo	3
+, -	Suma, resta	4

La *precedencia* se refiere al orden implícito de las operaciones cuando una expresión contiene dos o más operadores. El orden de operaciones se muestra para las siguientes expresiones:

$4 + 5 * 2$

Multiplicación, suma

$12 - 1 \text{ MOD } 5$

Módulo, resta

$-5 + 2$

Unario negativo, suma

$(4 + 2) * 6$

Suma, multiplicación

CONSTANTES NUMÉRICAS REALES

2.
+3.0
-44.2E+05
26.E5

Ejemplos de constantes numéricas reales válidas:

Las constantes numéricas reales se representan como reales decimales o reales codificados (hexadecimales). Un *real decimal* contiene un signo opcional seguido de un entero, un punto decimal, un entero opcional que expresa una fracción y un exponente opcional:

[signo]entero.[entero][exponente]

Se muestra la sintaxis para el signo y el exponente:

signo {+,-}

exponente E[{+,-}]*entero*

Se requieren, por lo menos, un dígito y un punto decimal.

- 
- **Reales codificados** Un real codificado representa a un número real en hexadecimal, utilizando el formato IEEE de punto flotante para los reales cortos. Por ejemplo, la representación binaria del número +1.0 decimal es:

0011 1111 1000 0000 0000 0000 0000 0000

- El mismo valor se codificaría como un real corto en lenguaje ensamblador de la siguiente manera:

3F800000r

CONSTANTES TIPO CARÁCTER

- Una *constante tipo carácter* es un solo carácter encerrado entre comillas sencillas o dobles. MASM almacena el valor en memoria como el código ASCII binario del carácter. Algunos ejemplos son:

'A'

"d"

CONSTANTES TIPO CADENA

- Una *constante tipo cadena* es una secuencia de caracteres (inclusive espacios) encerrados entre comillas sencillas o dobles:

'ABC'

'X'

"Buenas noches, Gracie"

'4096'

- Se pueden agregar comillas a la cadena, siempre y cuando se utilicen de la siguiente manera:

"Ésta no es una prueba"

'Diga "Buenas noches", Gracie'

PALABRAS RESERVADAS

- Las *palabras reservadas* tienen un significado especial en MASM, y sólo pueden usarse dentro de su contexto correcto. Hay distintos tipos de palabras reservadas:
 - Nemónicos de instrucciones, como MOV, ADD y MUL.
 - Directivas, las cuales indican a MASM cómo ensamblar programas.
 - Atributos, que proporcionan información acerca del tamaño y uso de las variables y operandos. Dos ejemplos son BYTE y WORD.
 - Operadores, que se utilizan en expresiones constantes.
 - Símbolos predefinidos como @data, que devuelven valores enteros constantes en tiempo de ensamblado.

DIRECTIVAS

- Una *directiva* es un comando incrustado en el código fuente, que el ensamblador reconoce y actúa en base a ésta. Las directivas no se llevan a cabo en tiempo de ejecución, mientras que las instrucciones sí. Las directivas pueden definir variables, macros y procedimientos. Pueden asignar nombres a los segmentos de memoria y realizar muchas otras tareas de mantenimiento relacionadas con el ensamblador. En MASM, las directivas no son sensibles a mayúsculas/minúsculas. MASM reconoce a **.data**, **.DATA** y a **.Data** como equivalentes.

■ El siguiente ejemplo nos ayudará a mostrar que las directivas no se llevan a cabo en tiempo de ejecución. La

Cada ensamblador tiene un conjunto distinto de directivas. Por ejemplo, TASM (Borland) y NASM (Netwide Assembler) comparten un subconjunto común de directivas con MASM. Por otro lado, el ensamblador GNU casi no tiene directivas en común con MASM.

DEFINICIÓN DE SEGMENTOS

- Una función importante de las directivas de ensamblador es definir las secciones, o *segmentos*, del programa. La directiva `.DATA` identifica el área de un programa que contiene variables:

`.data`

- La directiva `.CODE` identifica el área de un programa que contiene instrucciones:

`.code`

- La directiva `.STACK` identifica el área de un programa que guarda la pila en tiempo de ejecución, y establece su tamaño:

`.stack 100h`

INSTRUCCIONES

- Una *instrucción* es un enunciado que se vuelve ejecutable cuando se ensambla un programa. El ensamblador traduce las instrucciones en bytes de lenguaje máquina, para que la CPU los cargue y los lleve a cabo en tiempo de ejecución. Una instrucción contiene cuatro partes básicas:
 - ❖ Etiqueta (opcional).
 - ❖ Nemónico de instrucción (requerido).
 - ❖ Operando(s) (por lo general, son requeridos).
 - ❖ Comentario (opcional).
- Ésta es la sintaxis básica:

[etiqueta:] nemónico operando(s) [;comentario]
- Vamos a explorar cada parte por separado.

ETIQUETA

- Una *etiqueta* es un identificador que actúa como marcador de posición para las instrucciones y los datos. Una etiqueta que se coloca justo antes de una instrucción representa la dirección de esa instrucción. De manera similar, una etiqueta que se coloca justo antes de una variable representa la dirección de esa variable.
- *Etiquetas de datos* Una etiqueta de datos identifica la ubicación de una variable y proporciona una manera conveniente de hacer referencia a la variable dentro del código. El siguiente ejemplo define a una variable llamada cuenta:

cuenta DWORD 100

- El ensamblador asigna una dirección numérica a cada etiqueta. Es posible definir varios elementos de datos después de una etiqueta. En el siguiente ejemplo, la etiqueta arreglo define la ubicación del primer número (1024). Los demás números que le siguen en la memoria van inmediatamente después:

arreglo DWORD 1024, 2048

DWORD 4096, 8192

- *Etiquetas de código* Una etiqueta en el área de código de un programa (en donde se encuentran las instrucciones) debe terminar con un carácter de dos puntos (:). En este contexto, las etiquetas se utilizan como destinos de las instrucciones de saltos y de ciclos. Por ejemplo, la siguiente instrucción JMP (salto) transfiere el control a la ubicación marcada por la etiqueta llamada **destino**, con lo cual se crea un ciclo:

```
destino:
    mov    ax,bx
    ...
    jmp    destino
```

- Una etiqueta de código puede compartir la misma línea con una instrucción, o puede estar en una línea por sí sola:

```
L1:  mov    ax,bx
L2:
```

- Una etiqueta de datos no puede terminar con un signo de dos puntos. Los nombres de las etiquetas de datos deben ser únicos dentro del mismo archivo de código fuente; las etiquetas de código sólo deben ser únicas dentro del mismo procedimiento.

NEMÓNICO DE INSTRUCCIÓN

Un *nemónico de instrucción* es una palabra corta que identifica a una instrucción. En inglés, un *nemónico* es un dispositivo que ayuda a la memoria. De manera similar, los nemónicos de instrucciones en el lenguaje ensamblador, como mov, add y sub, proporcionan sugerencias acerca del tipo de operación que realizan:

mov	Mueve (asigna) un valor a otro
add	Suma dos valores
sub	Resta un valor de otro
mul	Multiplica dos valores
jmp	Salta a una nueva ubicación
call	Llama a un procedimiento

- **Operandos** Las instrucciones en lenguaje ensamblador pueden tener de cero a tres operandos, cada uno de los cuales puede ser un registro, un operando de memoria, una expresión constante o un puerto de E/S.
- Un *operando de memoria* se especifica mediante el nombre de una variable o mediante uno o más registros que contengan la dirección de una variable. El nombre de una variable indica la dirección de ésta, e instruye a la computadora para que haga referencia al contenido de la memoria en la dirección dada.

Ejemplo	Tipo de operando
96	Constante (<i>valor inmediato</i>)
2 + 4	Expresión constante
eax	Registro
cuenta	Memoria

operandos de ejemplo

EJEMPLOS DE INSTRUCCIONES EN LENGUAJE ENSAMBLADOR QUE TIENEN NÚMEROS VARIABLES DE OPERANDOS

`stc` ; activa la bandera Acarreo

`inc eax` ; suma 1 a EAX

En una instrucción con dos operandos, al primero se le llama el *destino* y al segundo el *origen*. En general, la instrucción modifica el contenido del operando de destino. Por ejemplo, en una instrucción MOV los datos se copian del origen al destino.

COMENTARIOS

- Los comentarios son un medio importante para que el escritor de un programa comunique información acerca de su funcionamiento a la persona que lee el código fuente. Con frecuencia se incluye la siguiente información en la parte superior del listado de un programa:
 - La descripción del propósito del programa.
 - Los nombres de las personas que crearon y revisaron el programa.
 - Las fechas de creación y revisión del programa.
 - Notas técnicas acerca de la puesta en marcha del programa.
- Los comentarios pueden especificarse en dos formas:
 - Comentarios de una sola línea, que empiezan con un carácter de punto y coma (;). El ensamblador ignora todos los caracteres que van después del punto y coma en la misma línea.
 - Comentarios de bloque, que empiezan con la directiva COMMENT y un símbolo especificado por el usuario. El ensamblador ignora todas las líneas subsiguientes de texto, hasta que aparezca el mismo símbolo especificado por el usuario. Por ejemplo,

```
COMMENT  !  
    Esta línea es un comentario.  
    Esta línea también es un comentario.  
!
```

```
COMMENT &  
    Esta línea es un comentario.  
    Esta línea también es un comentario.  
&
```

LA INSTRUCCIÓN NOP (NINGUNA OPERACIÓN)

- La instrucción más segura que podemos escribir se llama NOP (ninguna operación). Ocupa 1 byte de almacenamiento de programa y no hace nada. Algunas veces los compiladores y los ensambladores la utilizan para alinear el código con los límites de las direcciones pares.

En el siguiente ejemplo, la instrucción MOV genera tres bytes de código máquina. La instrucción NOP alinea la dirección de la tercera instrucción con un límite de doble palabra (múltiplo par de 4):

00000000	66 8B	C3	mov ax,bx	
00000003	90		nop	; alinea la siguiente instrucción
00000004	8B D1		mov edx,ecx	

Los procesadores IA-32 están diseñados para cargar código y datos con más rapidez de direcciones pares de doble palabra.

```

1  ; test.ams -- suma de dos numeros de 32 bits
2
3  .386
4  .model flat, stdcall
5  .stack 4096
6  ExitProcess proto, dwExitCode:dword
7
8  .data
9  sum DWORD ?
10
11 .code
12 main proc
13     mov eax, 7
14     add eax, 4
15     mov sum, eax
16     invoke ExitProcess, 0
17
18 main endp
19 end main
20
21

```

Se usa ; para comenzar un comentario

.data directiva que identifica el área del programa que contiene las variables.
.code directiva que identifica el área que contiene las instrucciones del programa.

main proc indica el comienzo del procedimiento principal

mov instrucción mnemotécnica. Mueve un valor a otro.

add instrucción mnemotécnica. Agrega dos valores
eax registro de propósito general
sum nuestra variable

Finaliza el procedimiento principal y el programa